Homework Assignment 1 : Deep Q-Network

In this homework you study **which ingredients make DQN work**, improve it with **Double DQN** and **Dueling DQN**, and test all variants on **an environment you build yourself**. Start from the DQN agent you wrote in Lab 3 (Exercise 1) and extend it.

Part	Content	Environments
Part 0	Presentation of the environments, preprocessing and hyperparameters	CartPole-v1, LunarLander-v3, ALE/Pong-v5
Exercise 1	Ablation study of DQN (hard / soft update, no replay, no target network)	CartPole-v1, LunarLander-v3, ALE/Pong-v5
Exercise 2	Double DQN and Dueling DQN	CartPole-v1, LunarLander-v3, ALE/Pong-v5
Exercise 3	Your own environment, built from scratch, in several versions	custom Gymnasium environment

General rules

- Every comparison uses **the same hyperparameters** and **the same seeds (0, 1, 2)**; only one setting changes at a time.
- For each run, report the **training score** (average return of the last 100 episodes for CartPole / LunarLander, last 20 games for Pong) and the **greedy score** ($\varepsilon = 0$, 20 new evaluation episodes, seeds never used during training).
- A difference between two variants is only meaningful if it is larger than the **min–max band across the 3 seeds**.
- Pong needs a GPU (Google Colab / Kaggle). If you cannot afford 10^6 steps per run, use the same smaller budget for every Pong variant and state it clearly in your report.

Part 0: The three environments

The same DQN agent is used on three tasks of increasing difficulty: a 4-dimensional control task, an 8-dimensional task with shaped and noisy rewards, and an Atari game played **from pixels**.

0.1 CartPole-v1 (classic control)

Element	Description
State $s \in \mathbb{R}^4$	cart position, cart velocity, pole angle, pole angular velocity
Actions $a \in \{0, 1\}$	push the cart to the left (0) or to the right (1)
Reward	+1 for every step the pole stays upright
Terminated	the pole falls or the cart leaves the track
Truncated	after 500 steps
Solved	average return ≥ 475 over 100 consecutive episodes

0.2 LunarLander-v3 (Box2D)

Element	Description
State $s \in \mathbb{R}^8$	x, y, velocities vx, vy, angle, angular velocity, left / right leg contact (0 or 1)
Actions $a \in \{0, 1, 2, 3\}$	do nothing, fire left engine, fire main engine, fire right engine
Reward	shaped: closer, slower and more level is better; +10 per leg on the ground; −0.3 per frame of main engine, −0.03 per frame of side engine; −100 for a crash, +100 for a safe landing
Terminated	the lander crashes, leaves the screen or comes to rest
Truncated	after 1000 steps
Solved	average return ≥ 200 over 100 episodes

You must install **box2d**: %pip install "gymnasium[box2d]" (it works with **Python 3.13** or lower).

0.3 ALE/Pong-v5 (Atari, from pixels)

Element	Description
Raw observation	RGB frame 210 × 160 × 3 at 60 frames per second
State $s \in \{0, \ldots, 255\}^{4\times84\times84}$	stack of the last 4 preprocessed frames (uint8)
Actions $a \in \{0, \ldots, 5\}$	NOOP, FIRE, RIGHT (up), LEFT (down), RIGHTFIRE, LEFTFIRE
Reward	+1 when the agent scores, −1 when the opponent scores
Terminated	one player reaches 21 points
Return range	[−21, +21]; a random agent gets about −20
Solved (our convention)	average return ≥ 18 over the last 20 games

Install the emulator with %pip install ale-py opencv-python, then call gym.register_envs(ale_py).

Atari preprocessing (Mnih et al., 2015), built with the Gymnasium wrappers AtariPreprocessing and FrameStackObservation:

Step	Wrapper / setting	Why
Frame skip = 4	AtariPreprocessing(frame_skip=4), env created with frameskip=1	the agent acts every 4 frames; 4× fewer decisions
Max-pooling of the last 2 frames	AtariPreprocessing	some Atari objects flicker
Grey scale + resize to 84 × 84	grayscale_obs=True, screen_size=84	colour is useless; 100 800 → 7 056 values

Step	Wrapper / setting	Why
Random no-ops at reset (≤ 30)	noop_max=30	different start states
Stack of the last 4 frames	FrameStackObservation(stack_size=4)	one frame shows no velocity; 4 frames make the state (almost) Markov
No sticky actions	repeat_action_probability=0.0	setting of the original paper
uint8 storage	replay memory in pre-allocated np.uint8 arrays, ÷ 255 inside the network	56 KB instead of 226 KB per transition
Reward clipping	the agent learns from sign(r); the true score is reported	same hyperparameters for every game

0.4 Q-networks

Environment	Architecture	Output
CartPole-v1	MLP 4 → 64 → 64 → 2, ReLU	linear, 2 Q-values
LunarLander-v3	MLP 8 → 128 → 128 → 4, ReLU	linear, 4 Q-values
ALE/Pong-v5	Nature CNN: conv 32 × 8×8 stride 4 → conv 64 × 4×4 stride 2 → conv 64 × 3×3 stride 1 → FC 512 (3 136 → 512), ReLU	linear, 6 Q-values (1 687 206 parameters)

0.5 Hyperparameters (reference DQN, hard update)

Hyperparameters	CartPole-v1	LunarLander-v3	ALE/Pong-v5
hidden_dim / network	64 (MLP)	128 (MLP)	Nature CNN
lr (Adam)	5×10^{-4}	5×10^{-4}	1×10^{-4}
gamma γ	0.99	0.99	0.99
buffer_size	100 000	100 000	100 000 (uint8)
batch_size	64	64	32
learn_every	1	4	1
learning_starts (warm-up)	64	64	10 000
target_update_every C	100	250	1 000
eps_start → eps_end	1.0 → 0.05	1.0 → 0.05	1.0 → 0.01
eps_decay_steps (env steps)	10 000	25 000	1000 000
Loss	MSE	MSE	Huber (smooth L1), grad norm clipped to 10
Training budget	300 episodes	800 episodes	1 000 000 env steps

Hyperparameters	CartPole-v1	LunarLander-v3	ALE/Pong-v5
Max steps per episode	500	1 000	one game (to 21)
target_score (solved)	475	200	18

Exercise 1: Ablation study of DQN on CartPole-v1, LunarLander-v3 and Pong

DQN (Mnih et al., 2015) adds two ingredients to deep Q-learning: **experience replay** and a **target network**. You measure what each one brings, and compare the two ways of updating the target network, on the three environments. Every variant uses **the same hyperparameters and the same seeds**; only one setting changes at a time.

Variant	What changes compared with the reference
DQN — hard update (reference)	replay memory (100 000 / 100 000 / 100 000), $\theta^- \leftarrow \theta$ every C gradient steps: C = 100 (CartPole), 250 (LunarLander), 1 000 (Pong)
DQN — soft update	$\theta^- \leftarrow \tau\,\theta + (1-\tau)\,\theta^-$ after every gradient step, $\tau = 0.01$
No experience replay	the minibatch is the B **most recent, consecutive** transitions (memory of size B): 64 (CartPole, LunarLander), 32 (Pong)
No target network	targets computed with the online network: $y = r + \gamma\,(1 - \text{terminated})\,\max_{a'} Q(s', a'; \theta)$

1. Add three switches to your **DQNAgent** without changing anything else:
   a. **target_update="hard" | "soft" with target_update_every** (C) and **tau** (τ).
   b. **use_replay=False**: the memory keeps only the last **batch_size** transitions. For Pong, **sample()** returns the B most recent transitions in order, and keep **learning_starts** = 10 000 steps of warm-up.
   c. **use_target_network=False: compute_targets** uses **q_net** instead of **target_net.**
2. Store, for every gradient step, the loss and the mean of $Q(s_i, a_i; \theta)$ over the minibatch (**agent.q_means**).
3. Train the 4 variants on each environment with **seeds 0, 1, 2** and the hyperparameters of Part 0 (4 variants × 3 seeds × 3 environments = 36 runs).
4. After training, evaluate each **greedy policy** ($\varepsilon = 0$) on **20 new episodes** (seeds 100 000 + i).
5. Report, for each environment:
   a. the learning curves (20-episode moving average, mean over seeds, shaded min–max band): hard vs soft, reference vs no replay, reference vs no target network;
   b. the curve of the predicted Q-values during training (with the line $1 / (1 - \gamma) = 100$ for CartPole);

Exercise 2: Double DQN and Dueling DQN on CartPole-v1, LunarLander-v3 and Pong

You extend the reference agent (DQN, hard update) with two classic improvements and compare four agents that share **everything** except the target or the network head.

Agent	Target y	Network
DQN	$r + \gamma\,(1 - \text{terminated})\ \max_{a'} Q(s', a'; \theta^-)$	standard head
Double DQN (van Hasselt et al., 2016)	$r + \gamma\,(1 - \text{terminated})\ Q(s', \text{argmax}_{a'} Q(s', a'; \theta); \theta^-)$	standard head
Dueling DQN (Wang et al., 2016)	DQN target	dueling head
Dueling Double DQN	Double DQN target	dueling head

Double DQN target: the online network θ **selects** the next action, the target network θ^- **evaluates** it:

$$a^*_i = \text{argmax}_{a'} Q(s'_i, a'; \theta), \quad y_i = r_i + \gamma\,(1 - \text{terminated}_i)\ Q(s'_i, a^*_i; \theta^-)$$

Dueling head: the network outputs a state value V(s) and advantages A(s, a), combined as:

$$Q(s, a; \theta) = V(s; \theta) + A(s, a; \theta) - (1 / |\mathcal{A}|)\ \Sigma_{a'} A(s, a'; \theta)$$

Environment	Shared body	Value stream V	Advantage stream A
CartPole-v1	Linear 4 → 64, ReLU	64 → 64 → 1	64 → 64 → 2
LunarLander-v3	Linear 8 → 128, ReLU	128 → 128 → 1	128 → 128 → 4
ALE/Pong-v5	the 3 Nature CNN convolutions (→ 3 136)	3 136 → 512 → 1	3 136 → 512 → 6

1. Add **double_dqn=True** to **compute_targets** (one extra forward pass of **q_net** on s′, under **torch.no_grad()**).
2. Write a class **DuelingQNetwork** for the MLP and one for the CNN, with the same **forward(state)** interface as **QNetwork;** add **dueling=True** to the agent.
3. Train the 4 agents on each environment with **seeds 0, 1, 2** and the hyperparameters of Part 0.
4. Evaluate each greedy policy on 20 new episodes
5. Plot the learning curves (mean over seeds, min–max band) and the predicted Q-values during training for the 4 agents.

Exercise 3: Your own environment, built from scratch

You design a Gymnasium environment, declare several versions of it (state representation, reward function, dynamics), and test **DQN**, **Double DQN** and **Dueling DQN** on each version. The agents of Exercises 1–2 must run **without modification**: only **env.observation_space** and **env.action_space** change.

3.1 Reference environment: TreasureGrid-v0

You may implement the reference environment below or propose your own (Snake, warehouse robot, traffic light, inventory, …) if it respects the same requirements; a personal environment is valued.

Element	Description
World	N × N grid (N = 6 by default) with walls, and dynamic 2 traps and 1 treasure
Actions a ∈ {0, 1, 2, 3}	up, down, left, right (moving into a wall = staying in place)
Terminated	the agent reaches the treasure or falls into a trap
Truncated	after 4 N^2 steps
Reset	random start cell; fixed or random treasure position (see versions)
Baselines	random agent and optimal agent (BFS shortest path)

3.3 Experiments

1. Propose several variants of your environment by modifying the reward function and state representation. For each variant, train and evaluate three agents: DQN, Double DQN, and Dueling DQN with **seeds 0, 1, 2**; choose and justify your hyperparameters (start from the CartPole column of Part 0), then keep them fixed.
2. Report the learning curves, the greedy score on 20 episodes, the success rate (treasure reached) and the mean episode length compared with the BFS optimum.

Deliverables: submission and grading

Submit one archive **NOM_Prenom_HW_DQN.zip** before the deadline announced in class.

File	Content
EX1_Ablation_<env>.ipynb (× 3)	ablation notebooks, executed, with outputs
EX2_DDQN_Dueling_<env>.ipynb (× 3)	Double / Dueling notebooks, executed, with outputs
EX3_CustomEnv.ipynb + treasure_grid.py	environment, **check_env**, experiments
*.pth	saved networks of the best seed of each agent
report.pdf (≤ 10 pages)	completed tables, curves, answers to the questions
video.mp4 (optional)	random agent vs best agent on Pong and on your environment

Part	Points
Part 0 + Exercise 1: switches, sanity checks, 36 runs, tables, curves	7
Exercise 2: Double DQN, Dueling networks, bias measurement	6
Exercise 3: environment, versions, experiments	6
Clarity and reproducibility (seeds, labelled plots)	1
Total	**20**